iOS Developer

PORTFOLIO

이한솔

PROFILE

이한솔 I iOS Developer

크래시 재발 0%를 9개월간 유지한 iOS 개발자입니다.

Objective-C 기반 서비스에서 Swift 전환을 병행하며, 앱 종료•백그라운드 전환 시 데이터 유실 문제를 해결하기 위해 생명주기 기반 자동 저장 로직을 설계하고, **출시 후 9개월간** 관련 이슈를 발생시키지 않았습니다. SwiftUI와 Clean Architecture 기반 개인 앱을 **기획부터 앱스토어 배포**까지 주도하며, 서버 개발자와 협업해 설계-구현-운영 전 과정을 경험하며 서비스를 완성했습니다. 이후 **베타 테스트(13명, 1주)**를 통해 사용자 피드백을 반영해 UX를 개선하고 앱스토어에 배포했습니다. **실시간 메시징 서비스 개발 경험을 바탕으로 소켓 기반 실시간 통신에 강점을 가지고 있습니다.**

기술 스택

Language	Swift, Objective-C	**Database**	Realm, UserDefaults
UI	UIKit, SwiftUI	**Platform**	Firebase (Crashlytics, FCM), WidgetKit, KeyChain
Concurrency	Combine, async/await	**Architecture**	MVC, MVVM, Clean Architecture
Network	REST API, TCP/IP Socket	**AI Tools**	Claude Code

학력

한국방송통신대학교 2026/02 ~ 재학중

컴퓨터과학과 3학년 편입

내일배움캠프 2023/06 ~ 2023/11

KDT 실무형 iOS 개발자 과정

• 4인 팀 리드, 앱스토어 출시 1건

강원관광대학교 2014/03 ~ 2016/12

관광계열 (카지노과) 전문 학사 졸업

WORK EXPERIENCE

(주) 유씨웨어 I iOS Developer

2024/10 ~ 재직중

기업용 실시간 협업 메신저 — B2B UCWORKS 6.0

주요 개발 기능

펑 채팅

메시지 만료 기한 설정

B2B 보안 요구 대응
TCP/IP Socket 기반 실시간 채팅
GCD 타이머 + Realm 마이그레이션

채팅 임시 저장

자동 저장 및 복원

이모티콘·멘션 포함
완전 복원 구조 단독 설계

쪽지 라벨링

수동·자동 분류 체계

라벨 CRUD, 수동 라벨링
키워드·발신자 기반 자동 라벨링

주요 기여

Crashlytics 도입

Nil 참조 크래시 원인 파악 및 해결

동일 이슈 재발 0건 (9개월 +)

Obj-C → Swift 점진적 전환

로그인 등 주요 화면 Swift 전환, ObjC·Swift 혼용 구조 유지

서비스 중단 0건

GS 인증 대응

담당 개발자로 보안 취약점 20건+ 분석·수정 주도

GS인증 취득 완료

5개+ 고객사 오픈 및 고도화 사업 담당

고객사별 기능 커스터마이징·QA 전 과정 주도

오픈 후 크리티컬 이슈 0건

WORK EXPERIENCE

채팅 만료 기한 설정 기능

01 기능 개요

배경

고객사 보안 요구로 메시지·파일 만료 기능 신규 스펙 접수
iOS 클라이언트 설계 및 구현 전담

해결

서버 동기화 + 클라이언트 선제 판단 이중 구조 설계
(GCD DispatchSourceTimer, Realm Migration 적용)

성과

2개 고객사 배포, **관련 클레임 0건**
클라이언트 만료 파일 UI 처리 - 다운로드 클릭 시 API 호출 차단, **불필요한 네트워크 요청 제거**

02 기능 흐름

채팅 메시지 롱 프레스 - 만료 기한 설정 UI
UIKit + ActionSheet

↓

서버 동기화, Realm 만료 필드 저장
Realm Migration + TCP Packet

↓

DispatchSourceTimer 주기적 만료 체크

↓

만료 시 UI 차단 + API 호출 차단

03 핵심 문제 해결

• 백그라운드 복귀 시 타이머 정합성

문제

앱이 백그라운드에 있는 동안 GCD 타이머가 suspend되어 만료 시점을 놓칠 수 있음

해결

UIApplicationWillEnterForeground 시점에 expirePendingMap 전체 현재 시간 기준으로 즉시 재검사

결과

백그라운드 체류 시간과 무관하게 만료 상태 정확히 반영

• Realm 마이그레이션 무손실 처리

문제

기존 사용자 DB에 만료 관련 필드 4개를 추가해야 하며, 마이그레이션 실패 시 앱 크래시 발생

해결

Schema v19로 마이그레이션 블록 작성,
기존 데이터에 빈 문자열 디폴트값 설정

결과

기존 사용자 데이터 무손실 마이그레이션, 관련 크래시 0건

WORK EXPERIENCE

채팅 임시 저장 및 복원 기능

01 기능 개요

배경

채팅방 이탈 · 앱 백그라운드 전환 시 작성 중이던 메시지가 유실되는 **문의 다수 접수**, 사용자가 긴 메시지를 다시 입력해야 하는 불편 발생

해결

채팅방별 고유 키(userId + roomKey)로 UserDefaults에 저장 대상 데이터를 분리 저장, 채팅방 재진입 시 자동 복원

성과

사용자 재입력 부담 제거 및 사용성 향상
출시 후 약 **9개월간 관련 크래시/CS 이슈 0건 유지**

02 기능 흐름

채팅방 이탈 / 백그라운드 진입 — 임시 저장
viewWillDisappear + NSNotification

↓

UserDefaults 저장 (PrivateStorage)
draftText · draftEmoji · draftNomiID · draftNomiName

↓

채팅방 재진입 — 자동 복원
viewWillAppear에서 키 조회 + AttributedString 재조합

↓

메시지 전송 성공 시 — 임시 데이터 삭제
resetSaveDraftText 호출

03 핵심 문제 해결

• 커스텀 이모티콘 직렬화

문제

커스텀 이모티콘이 NSTextAttachment(이미지)로 삽입되어 일반 String 저장 시 유실

해결

NSAttributedString의 커스텀 attribute순회,
UUID→태그 변환 후 저장 / 복원 시 태그→이미지 역변환

결과

이모티콘·텍스트 혼합 메시지 완전 복원

• 멘션 대상 컨텍스트 유실

문제

텍스트만 저장하면 멘션 수신인 정보가 유실

해결

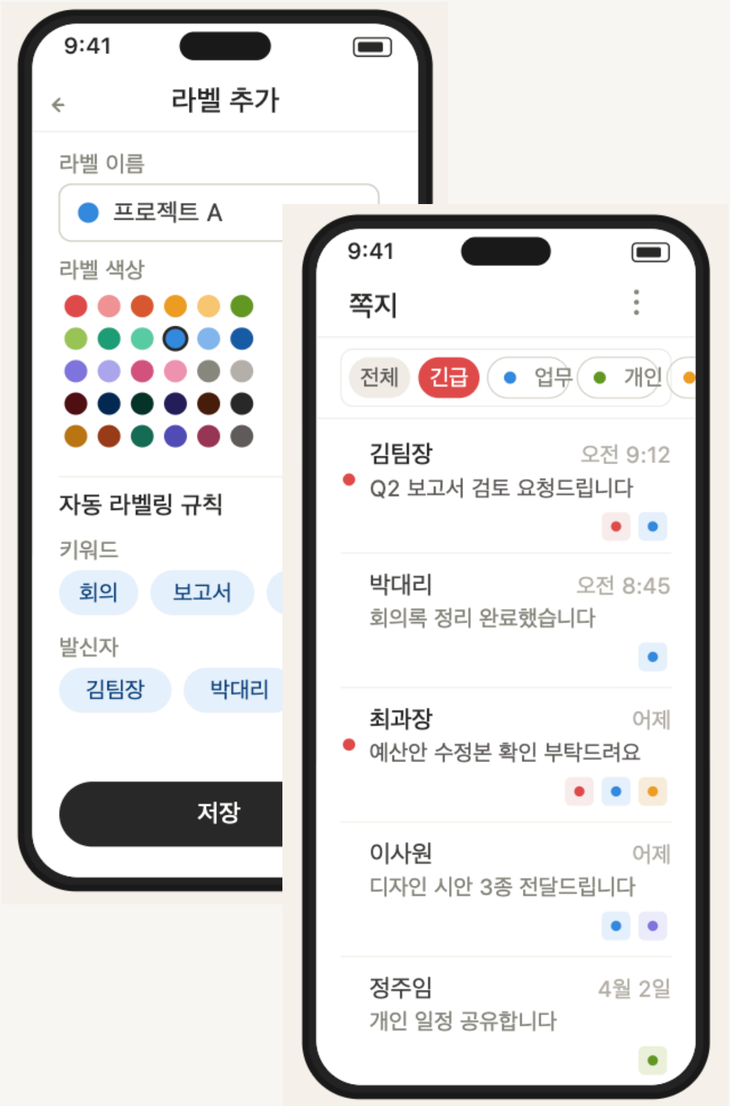
멘션 대상 ID·이름을 별도 키로 분리 저장,
복원 시 ChatMstInfo에 재주입

결과

멘션 대상 포함 완전한 입력 상태 복원

WORK EXPERIENCE

쪽지 라벨• 라벨링 기능



01 기능 개요

배경

쪽지(사내 메일)가 누적될수록 원하는 쪽지를 찾기 어렵고, 별도 분류 체계가 없어 전체 목록에서 직접 탐색해야 함

해결

라벨 CRUD + 자동 라벨링 체계를 도입하고, Realm 기반 로컬 저장으로 라벨별 필터링·검색 기능 구축

성과

쪽지 검색 속도 체감 시간 단축 및 **사용자 피드백 긍정 반응**
자동 라벨링 기능으로 사용자 수동 분류 부담 감소

02 기능 흐름

라벨 생성 — CRUD + 순서 관리
UIKit + Drag & Drop 재정렬

↓

라벨 지정 — 수동 / 자동 라벨링
수동: 바텀시트 멀티 선택 | 자동: 키워드·발신자 Rule 매칭

↓

Realm 저장 — 라벨 데이터 로컬 persist
LabelDataObject + MessageDataObject 연동

↓

라벨별 필터링 · 검색
수신/발신 쪽지 라벨 필터 + 서버 동기화

03 핵심 문제 해결

• 자동 라벨링 대상 분류

문제

수신·발신 쪽지 구분 및 본인 발신 쪽지 처리 시
동일 Rule이 중복 적용되어 라벨 누락 또는 중복 지정 발생

해결

수신/발신 여부와 발신자 ID 기준으로 분기 처리,
본인 발신 케이스 별도 예외 처리

결과

모든 쪽지 유형에서 정확한 라벨 자동 지정

• 자동 라벨링 조건 중복 매칭

문제

키워드 Rule과 발신자 Rule이 동시에 매칭될 때
중복 라벨 지정 및 우선순위 문제

해결

매칭된 모든 Rule의 라벨을 합산 적용, 중복 제거 후
labelCodes에 반영

결과

복수 Rule 동시 매칭 시에도 정확한 라벨 지정

PERSONAL PROJECT

FaithConnect

익명 기반 기도 커뮤니티 서비스

2025/11 ~ 진행중 | 개인 프로젝트

App Store | GitHub

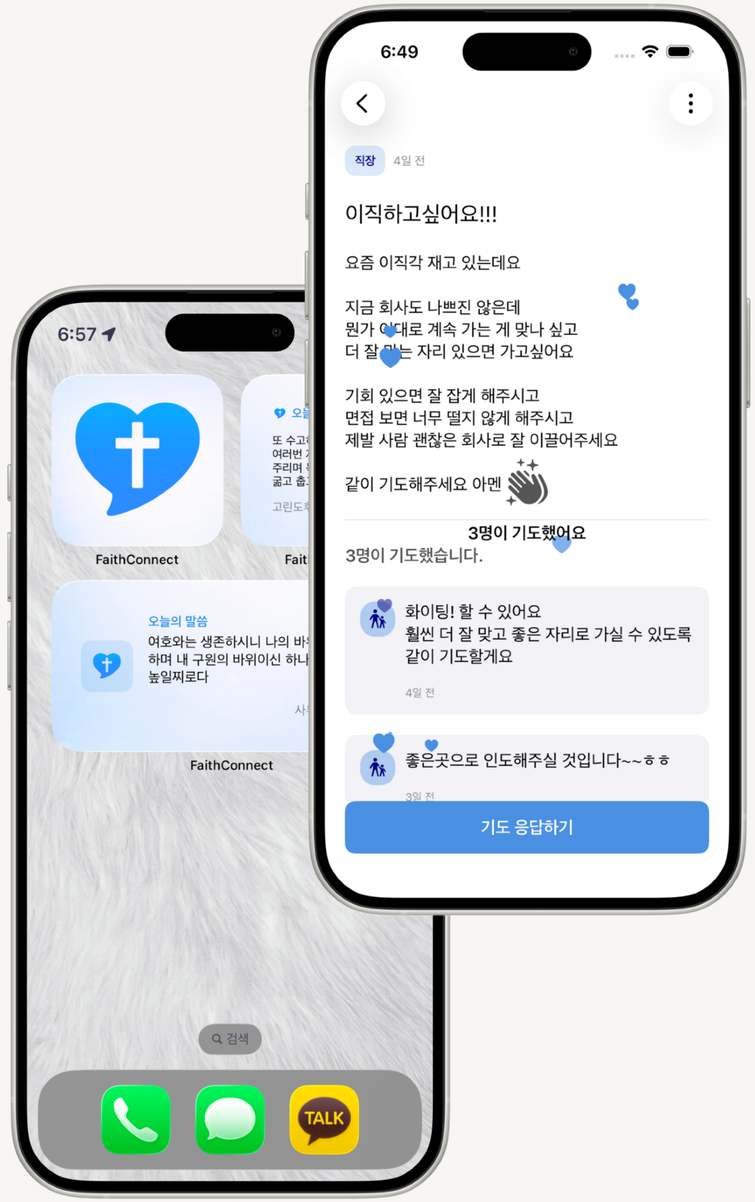
기술 스택

- SwiftUI
- Firebase
- Combine
- async/await
- KeyChain
- WidgetKit

역할

클라이언트 파트 전 과정 주도 (서버 개발 협업)

13명	0건	40개
베타 테스터	세션 이슈	Unit Test



01 프로젝트 진행 사유

배경

실명 노출 부담 없이, 교회 밖에서도 많은 사람들과 기도를 나눌 수 있는 익명 커뮤니티의 필요성을 느낌

해결

익명성 기본 전제 + Clean Architecture 도입으로 향후 기능 확장에 용이한 구조 설계

02 아키텍처

Presentation Layer
SwiftUI + ViewModel (@MainActor, @Published)

↓

Domain Layer
Use Cases + Protocols + Entities (DTO → Entity 변환)

↓

Data Layer
Repository + API + Keychain Storage

03 핵심 문제 해결

JWT 레이스 컨디션

문제 동시 API 요청 시 RefreshToken 중복 갱신 → 두 번째 요청부터 인증 실패

해결 RefreshTask로 진행 중인 갱신을 공유, 후속 요청은 같은 Task를 await

결과 세션 이슈 0건

Combine 이벤트 기반 실시간 UI 동기화

문제 홈·내기도·상세가 동일 데이터 표시 — 매번 서버 재조회 시 UX 저하

해결 PassthroughSubject로 이벤트 발행, ViewModel이 구독해 로컬 배열 직접 수정

결과 서버 재조회 없이 즉시 UI 반영

FCM 푸시 알림 3가지 앱 상태 분기

문제 Foreground·Background·Terminated 각 상태별 타이밍 달라 화면 이동 누락

해결 DeepLinkManager의 pendingDestination으로 큐잉, 단일 매니저로 수렴

결과 모든 앱 상태에서 동일하게 화면 이동

04 KPT 회고

Keep

- Clean Architecture 도입으로 기능 추가 시 수정 범위 최소화
- Protocol DI로 Mock 데이터로 UnitTest 가능 구조 확보
- 베타 피드백을 1주 내 UX에 빠르게 반영

Problem

- 기능 추가로 서버 API 스펙이 예상치 못하게 변경, 기존 구현 수정 비용 발생
- 테스트 커버리지 목표를 초반에 설정하지 않아 후반에 몰아서 작성
- 베타 인원이 적어 다양한 OS 버전·디바이스 검증 부족

Try

- 기능 추가 시 API 스펙 변경 가능성을 미리 고려하고, 변경에 유연한 DTO 구조 설계
- 프로젝트 초반부터 테스트 커버리지 목표 설정
- 베타 인원 확대로 다양한 OS 버전·디바이스 커버

Thank you for reading

언제든 편하게 연락주세요

📧 **Email**

ho20128@naver.com

🔗 **GitHub**

github.com/Leehan-sol

📝 **Velog**

velog.io/@ho20128

📞 **Phone**

+82-10-9171-0128